

Fernando Lopez

872-305-7616 | fernando@pz.dev | flope8@uic.edu | [linkedin.com/in/fernando-lopez-dev](https://www.linkedin.com/in/fernando-lopez-dev) | github.com/FernandoLpz0911

EDUCATION

University of Illinois Chicago (UIC) – B.S. in Computer Science

Expected: May 2027

TECHNICAL SKILLS

Languages: Python, Dart, Swift, C++, Java, JavaScript, Go, SQL

Frameworks & Libraries: Flutter, FastAPI, Django, React, XGBoost, scikit-learn, CoreML, AVFoundation, Pytest, JUnit5, XCTest

Cloud & DevOps: Google Cloud Platform (Cloud Run, Cloud Build), Firebase, Docker, GitHub Actions

Tools & Databases: Git, Linux, Vim, PostgreSQL, SQLite, Figma, Firebase (Firestore / Storage / Auth / Crashlytics)

Other: MVVM / Provider state management, REST APIs, CI/CD pipelines, Agile

EXPERIENCE

Sanchez Research Lab, UIC – Undergraduate Research Engineer

Jan 2026 – Present

Voxylex AI — Clinical Speech Research Platform (Flutter, Firebase, Cloud Run, Dart, GitHub Actions)

- Built the multi-app clinical platform for UIC's oropharyngeal research: a patient-facing mobile app (iOS/Android), a Windows physician desktop dashboard, and a web admin console, all on one Firebase backend.
- Designed a multi-tenant Firestore schema with denormalized reverse-lookup documents and atomic batch writes, so the physician, patient and disease-list collections cannot drift out of sync.
- Implemented the multi-window IPC security layer for the desktop dashboard: a per-session ephemeral capability secret gates every child-window API call, and the main window brokers them, validating secrets, refreshing tokens and retrying on 401/403.
- Engineered atomic patient registration with Firebase Auth rollback: a secondary App instance isolates account creation from the physician's session, and a failed Firestore write deletes the new Auth user rather than leaving an orphaned credential behind.
- Centralized async error telemetry behind a generic `errorCheck<T>` wrapper with per-session deduplication and dual-mode output, console in development and Firestore in production, with Crashlytics handling crashes.
- 600+ unit tests across 11 suites using fake backend injection and idempotency verification instead of mocking our own code, including a 2,957-line desktop suite; GitHub Actions runs them with `flutter analyze --fatal-infos`, emulator integration tests and signed APK/AAB builds.
- Shipped 7-language localization (English, Spanish, Catalan, French, Russian, Vietnamese, Chinese), timezone-aware notification scheduling, NTP server-time sync so streaks cannot be spoofed from the client, and Firebase App Check (PlayIntegrity + AppAttest).

Orosense — Real-Time Oral Motor Assessment (Swift, CoreML, TrueDepth, Flutter, Firebase)

- Wrote the CoreML inference wrapper in Swift from scratch: input dimensions read from model metadata, letterbox resize through `CIImage` on the GPU, and raw `[1, 17, N]` tensor output parsed for float32 and float16 layouts in either dimension order.
- Implemented NMS in Swift by hand, with IoU, confidence thresholding, and tongue-tip visibility scoring across the surviving candidates.
- Fused the TrueDepth depth stream with video frames to compute tongue protrusion in real millimeters, unprojecting 2D keypoints and their depth through the camera intrinsic matrix `(fx, fy, cx, cy)` into camera-space vectors.
- Built self-updating model distribution over the HuggingFace REST API: a Swift actor checks lastModified timestamps, downloads .mlpackage archives, compiles them on device, caches by version, and degrades quietly when offline.
- Cut depth storage from roughly 160–200 MB per assessment by cropping each frame at capture to a nose-down window sized in inter-commissure distance, and exported a pivoted CSV per patient with 35+ metrics per task.
- Picked raw DEFLATE over LZFS for the depth archives even though LZFS is faster on-device and about 10–15% smaller, because the consumers are a browser viewer and Python. Wrote it up as an ADR and pinned the codec behind 97 XCTest cases.

GMPLabeler — Browser-Based ML Dataset Annotation Tool (JavaScript, WebGPU, WGSL, MediaPipe)

- Built a zero-dependency browser annotation tool for YOLO training data: webcam and video capture, auto-detection of tongue tip and lip landmarks, click-to-correct editing. No build step, no framework.

- Wrote a WGSL compute shader dispatched in 16×16 workgroups that classifies every mouth-ROI pixel as tongue or not on the GPU, managing the whole WebGPU lifecycle: buffer allocation, bind groups, command encoding, readback, explicit destruction per frame.
- Pipelined MediaPipe's 478-point face mesh with the async GPU detector behind a guard flag so dispatches never overlap, and exported to both YOLO Pose and YOLO Detect with train/val split and data.yaml.

Dulce Tentaciones – *Freelance Software Engineer*

Jun 2025 – Aug 2025

- Directed the full SDLC for a local bakery startup, delivering a responsive React application from requirements to deployment.
- Engineered a marketing pipeline using Brevo and EmailJS capable of processing 1,000+ monthly customer orders without manual intervention.
- Integrated Sanity CMS to enable non-technical content updates, reducing developer maintenance time.

Compudopt Inc. – *Program Teacher*

Aug 2024 – Feb 2025

- Translated programming concepts into accessible K-8 curriculum covering loops, control flow, and variable types for 20+ students per semester.
- Directed hands-on engineering workshops using Arduino microcontrollers and circuit logic to build early embedded systems literacy.

PROJECTS

Interrupted Execution Coherence (IEC) – *Paper draft, sole author (Python, llama.cpp)*

Apr – Jun 2026

- Found that llama.cpp validates a restored KV-cache slot against its cell metadata (sequence ids, positions, used flags) and never reads the attention values, so a slot with corrupted float16 K/V tensors and byte-identical metadata restores cleanly: no rejection, no cold-start fallback, no error.
- Wrote the tooling against the raw binary slot format: auto-detection of the tensor block (v_trans, n_layer) so transposed-V models like Gemma-2 parse without a known layer count, targeted byte-level corruption, and sweep drivers that verify every save and restore before a trial counts.
- Built the detection that closes the gap, HMAC-SHA256 over the tensor region plus finite and range checks at restore, and scored roughly 1,500 trials behind a validity anchor that fails loudly when a baseline drifts.

Workout AI – *Personal Project*

Apr 2026 – Present

Full-stack fitness application — Flutter mobile + Python/XGBoost backend + FastAPI on Google Cloud Run

- Built end-to-end ML fitness application: Flutter mobile frontend with Provider state management, Python/XGBoost prediction backend, and FastAPI REST API deployed on Google Cloud Run via Docker and Cloud Build CI/CD.
- Implemented offline-first recommendation engine in Dart (231 lines) that mirrors the full Python/XGBoost pipeline — 1RM formula selection by rep range (Brzycki/Epley/Mayhew), momentum calculation, rep consistency, and double-progression rules — with no network required.
- Engineered the CSV pipeline from FitNotes exports: regex warm-up and drop-set filtering, fingerprint-based duplicate detection, category classification across 8 strength groups, and feature engineering on top.

ProphetHacks 2026 — Trading Track – *Python, GCP, Docker, SQLite, pytest*

May 2026

- Built and deployed an autonomous prediction-market trading agent running unattended for 14 days on GCP Compute Engine (E2), executing trades on live Kalshi markets every 15 minutes across 1,344 ticks.
- Replaced the vendor SDK's LLM-based action stage with a deterministic 8-step position sizer, cutting roughly 40% of per-tick latency and one redundant LLM call per market per tick across the full evaluation window.
- Implemented a thread-safe SQLite spend ledger with a configurable \$180 USD kill switch: once LLM costs cross the threshold, the forecast stage degrades to market mid-price (zero edge, zero LLM spend) automatically.
- Full risk pipeline enforcing 8 overlapping constraints per tick: per-market notional cap, gross exposure cap, max open positions (30), max trades/tick (20), per-category cap, take-profit/stop-loss, and position-flip-as-sell logic.

LEADERSHIP & ACTIVITIES

Women in Computer Science — Aspire To Inspire – *Mentor*

Sep 2025 – Present

- Led a 4-person agile team building a real-time chat application in Django and React.
- Mentored 4 students through full-stack development and end-to-end deployment of personal web portfolios.